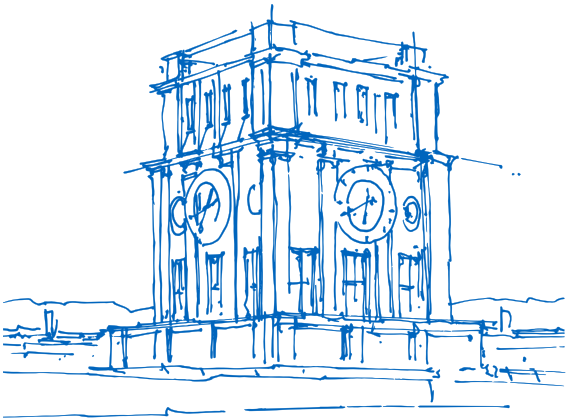
Einführung in die Rechnerarchitektur

Zentralübung #2: Einführung in RISC-V Assembly

Dominic Prinz & Jakob Schäffeler

Lehrstuhl für Rechnerarchitektur & Parallele Systeme
Technische Universität München

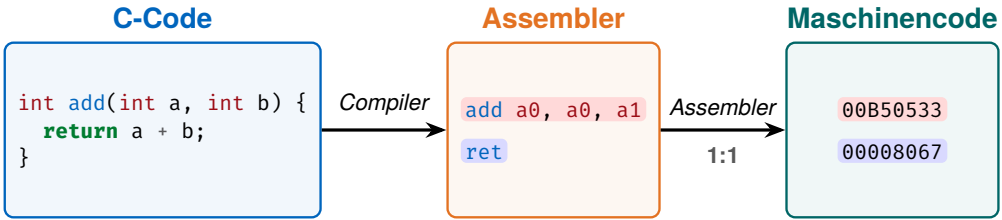


RISC-V Assembly Roadmap

- Heute: Einführung in QtRVSim und einfache Assembly-Programme
- Nächste Woche: Unterprogramme, Speicherzugriffe und Sprünge in RISC-V
- Übernächste Woche: Komplexe Programme mit Rekursion und Calling Convention

Von Code zu einer ausführbaren Datei

- C-Code oder ähnliches nicht direkt ausführbar
- Ziel: Ausführbare Binärdatei mit Maschinencode
- Assembler: Niedrigste Abstraktionsebene vor Maschinencode mit primitiven Befehlen



RISC-V Befehlssatz

- Offener, modularer Befehlssatz: Basis + optionale Erweiterungen
- Basis: `RV32I`, `RV64I`, `RV128I`
  - Zahl = Registerbreite in Bit
  - `I` = Integer-Grundbefehle (ca. 40 Befehle)
- Optionale Erweiterungen: M(ul), F(loating), A(tomic), V(ector), ...
- Wir verwenden `RV32IM`
- Details können im RISC-V Instruction Set Manual Volume I nachgelesen werden

RISC-V Register

- 32-Bit große Speicherzellen
- Innerhalb des Prozessors
- Konvention für Verwendung
- `x0` bis `x31` und Program Counter

Name	Register Number	Use
`zero`	`x0`	Constant value 0
`ra`	`x1`	Return address
`sp`	`x2`	Stack pointer
`gp`	`x3`	Global pointer
`tp`	`x4`	Thread pointer
`t0–2`	`x5–7`	Temporary registers
`s0/fp`	`x8`	Saved register/Frame pointer
`s1`	`x9`	Saved register
`a0–1`	`x10–11`	Function arguments/Return values
`a2–7`	`x12–17`	Function arguments
`s2–11`	`x18–27`	Saved registers
`t3–6`	`x28–31`	Temporary registers

RISC-V Befehle

- Befehle müssen nicht auswendig gelernt werden
- Funktion kann aus Tabelle abgelesen werden
- Hieraus können auch Opcodes abgelesen werden

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	`lb rd, imm(rs1)`	load byte	rd = SignExt([Address]$_{7:0}$)
0000011 (3)	001	–	I	`lh rd, imm(rs1)`	load half	rd = SignExt([Address]$_{15:0}$)
0000011 (3)	010	–	I	`lw rd, imm(rs1)`	load word	rd = [Address]$_{31:0}$
0000011 (3)	100	–	I	`lbu rd, imm(rs1)`	load byte unsigned	rd = ZeroExt([Address]$_{7:0}$)
0000011 (3)	101	–	I	`lhu rd, imm(rs1)`	load half unsigned	rd = ZeroExt([Address]$_{15:0}$)
0010011 (19)	000	–	I	`addi rd, rs1, imm`	add immediate	rd = rs1 + SignExt(imm)
0010011 (19)	001	0000000*	I	`slli rd, rs1, uimm`	shift left logical immediate	rd = rs1 ≪ uimm
0010011 (19)	010	–	I	`slti rd, rs1, imm`	set less than immediate	rd = (rs1 < SignExt(imm))
0010011 (19)	011	–	I	`sltiu rd, rs1, imm`	set less than immediate unsigned	rd = (rs1 < SignExt(imm))
0010011 (19)	100	–	I	`xori rd, rs1, imm`	xor immediate	rd = rs1 ^ SignExt(imm)
0010011 (19)	101	0000000*	I	`srli rd, rs1, uimm`	shift right logical immediate	rd = rs1 ≫ uimm
0010011 (19)	101	0100000*	I	`srai rd, rs1, uimm`	shift right arithmetic immediate	rd = rs1 ⋙ uimm
0010011 (19)	110	–	I	`ori rd, rs1, imm`	or immediate	rd = rs1 | SignExt(imm)
0010011 (19)	111	–	I	`andi rd, rs1, imm`	and immediate	rd = rs1 & SignExt(imm)
0010111 (23)	–	–	U	`auipc rd, upimm`	add upper immediate to PC	rd = {upimm, 12'b0} + PC
0100011 (35)	000	–	S	`sb rs2, imm(rs1)`	store byte	[Address]$_{7:0}$ = rs2$_{7:0}$
0100011 (35)	001	–	S	`sh rs2, imm(rs1)`	store half	[Address]$_{15:0}$ = rs2$_{15:0}$
0100011 (35)	010	–	S	`sw rs2, imm(rs1)`	store word	[Address]$_{31:0}$ = rs2
0110011 (51)	000	0000000	R	`add rd, rs1, rs2`	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	`sub rd, rs1, rs2`	sub	rd = rs1 − rs2
0110011 (51)	001	0000000	R	`sll rd, rs1, rs2`	shift left logical	rd = rs1 ≪ rs2$_{4:0}$
0110011 (51)	010	0000000	R	`slt rd, rs1, rs2`	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	`sltu rd, rs1, rs2`	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	`srl rd, rs1, rs2`	shift right logical	rd = rs1 ≫ rs2$_{4:0}$
0110011 (51)	101	0100000	R	`sra rd, rs1, rs2`	shift right arithmetic	rd = rs1 ⋙ rs2$_{4:0}$
0110011 (51)	110	0000000	R	`or rd, rs1, rs2`	or	rd = rs1 | rs2
0110011 (51)	111	0000000	R	`and rd, rs1, rs2`	and	rd = rs1 & rs2
0110011 (51)	000	0000001	R	`mul rd, rs1, rs2`	multiply	rd = (rs1 * rs2)$_{31:0}$
0110011 (51)	001	0000001	R	`mulh rd, rs1, rs2`	multiply high (signed)	rd = (rs1 * rs2)$_{63:32}$
0110011 (51)	010	0000001	R	`mulhsu rd, rs1, rs2`	multiply high (signed/unsigned)	rd = (rs1 * rs2)$_{63:32}$
0110011 (51)	011	0000001	R	`mulhu rd, rs1, rs2`	multiply high (unsigned)	rd = (rs1 * rs2)$_{63:32}$
0110111 (55)	–	–	U	`lui rd, upimm`	load upper immediate	rd = {upimm, 12'b0}
1100011 (99)	000	–	B	`beq rs1, rs2, label`	branch if =	if (rs1 == rs2) PC = BTA
1100011 (99)	001	–	B	`bne rs1, rs2, label`	branch if ≠	if (rs1 ≠ rs2) PC = BTA
1100011 (99)	100	–	B	`blt rs1, rs2, label`	branch if <	if (rs1 < rs2) PC = BTA
1100011 (99)	101	–	B	`bge rs1, rs2, label`	branch if ≥	if (rs1 ≥ rs2) PC = BTA
1100011 (99)	110	–	B	`bltu rs1, rs2, label`	branch if < unsigned	if (rs1 < rs2) PC = BTA
1100011 (99)	111	–	B	`bgeu rs1, rs2, label`	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA
1100111 (103)	000	–	I	`jalr rd, rs1, imm`	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4
1101111 (111)	–	–	J	`jal rd, label`	jump and link	PC = JTA, rd = PC + 4

*Encoded in instr$_{31:25}$, the upper seven bits of the immediate field

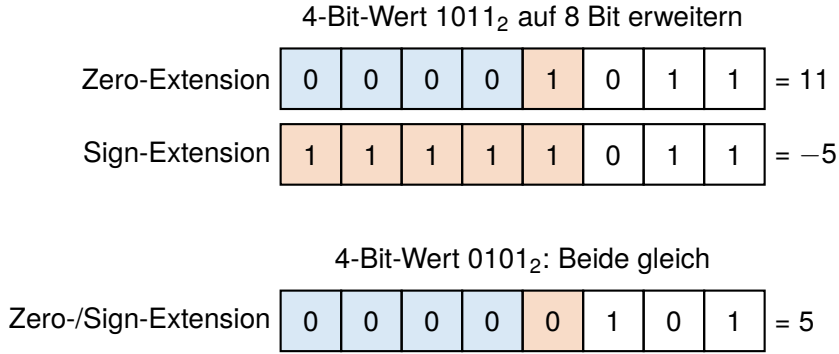
Addition und Subtraktion

- Kein `subi` verfügbar
- Negative Immediates bei `addi` möglich

op	funct3	funct7	Type	Instruction	Description	Operation
0010011 (19)	000	–	I	`addi rd, rs1, imm`	add immediate	`rd = rs1 + SignExt(imm)`
0110011 (51)	000	0000000	R	`add rd, rs1, rs2`	add	`rd = rs1 + rs2`
0110011 (51)	000	0100000	R	`sub rd, rs1, rs2`	sub	`rd = rs1 - rs2`

- `addi` kann zum Laden von Konstanten verwendet werden
- `addi` kann aber nur bis zu 12-Bit-Konstanten laden

Zero- und Sign-Extension



- Zero-Extension: Neue Bits mit 0 füllen (unsigned)
- Sign-Extension: Neue Bits mit dem Vorzeichenbit füllen (signed)

RISC-V Befehlstypen

- Warum nur 12-Bit Konstanten?
- Befehle werden in unterschiedliche Typen eingeteilt

31:25	24:20	19:15	14:12	11:7	6:0	
funct7	rs2	rs1	funct3	rd	op	**R-Type**
$imm_{11:0}$		rs1	funct3	rd	op	**I-Type**
$imm_{11:5}$	rs2	rs1	funct3	$imm_{4:0}$	op	**S-Type**
$imm_{12,10:5}$	rs2	rs1	funct3	$imm_{4:1,11}$	op	**B-Type**
$imm_{31:12}$				rd	op	**U-Type**
$imm_{20,10:1,11,19:12}$				rd	op	**J-Type**

Laden von großen Konstanten

- `lui rd, upimm`
  - `upimm` hat 20 Bit
  - `upimm` wird zuerst um 12 nach links geshiftet
  - Dann die unteren 12 Bit mit Nullen überschrieben
  - Anschließend Ergebnis nach `rd` geschrieben
- Zum Laden von großen Konstanten zuerst `lui`, dann `addi`
- **Achtung:** Sign-Extension von `addi` kann zu unerwarteten Ergebnissen führen (siehe Übungsblatt 2)

Multiplikations- und Divisionsbefehle

- Teilweise mehrere Varianten je nach Signedness

op	funct3	funct7	Type	Instruction	Description	Operation
0110011 (51)	000	0000001	R	`mul rd, rs1, rs2`	multiply	$\texttt{rd} = (\texttt{rs1} * \texttt{rs2})_{31:0}$
0110011 (51)	001	0000001	R	`mulh rd, rs1, rs2`	multiply high (signed)	$\texttt{rd} = (\texttt{rs1} * \texttt{rs2})_{63:32}$
0110011 (51)	010	0000001	R	`mulhsu rd, rs1, rs2`	multiply high (signed/unsigned)	$\texttt{rd} = (\texttt{rs1} * \texttt{rs2})_{63:32}$
0110011 (51)	011	0000001	R	`mulhu rd, rs1, rs2`	multiply high (unsigned)	$\texttt{rd} = (\texttt{rs1} * \texttt{rs2})_{63:32}$
0110011 (51)	100	0000001	R	`div rd, rs1, rs2`	divide (signed)	`rd = rs1 / rs2`
0110011 (51)	101	0000001	R	`divu rd, rs1, rs2`	divide (unsigned)	`rd = rs1 / rs2`
0110011 (51)	110	0000001	R	`rem rd, rs1, rs2`	remainder (signed)	`rd = rs1 % rs2`
0110011 (51)	111	0000001	R	`remu rd, rs1, rs2`	remainder (unsigned)	`rd = rs1 % rs2`

Multiplikations- und Divisionsbefehle

- Teilweise mehrere Varianten je nach Signedness
- 32 Bit · 32 Bit = bis zu 64 Bit ⇒ zwei Teilergebnisse
- Beispiel mit 4 Bit (Produkt 8 Bit): **gleiche Bits**, anders interpretiert

Bits	unsigned	signed
`1101 · 0010`	13 · 2 = 26 = `0001_1010`	(−3) · 2 = −6 = `1111_1010`
`1111 · 1001`	15 · 9 = 135 = `1000_0111`	(−1) · (−7) = 7 = `0000_0111`

- Untere Hälfte unabhängig von Signedness ⇒ nur ein Befehl: `mul`
- Obere Hälfte hängt von den Vorzeichen ab ⇒ `mulh`, `mulhu`, `mulhsu`
- Aufgabe des Assembler-Programmierers, die passenden Instruktionen zu wählen

Wann reicht `mul`?

- `mul` liefert nur die unteren 32 Bit
- Richtiges Ergebnis, wenn es in 32 Bit passt

Bits	gerechnet als	untere 4 Bit
`0011 · 0101 = 0000_1111`	unsigned	$3 \cdot 5 = 15 =$ `1111` ✓
`1101 · 0010 = 0001_1010`	signed	$(-3) \cdot 2 = -6 =$ `1010` ✓
`1111 · 1001 = 1000_0111`	signed	$(-1) \cdot (-7) = 7 =$ `0111` ✓

Wann reicht `mul`?

- `mul` liefert nur die unteren 32 Bit
- Richtiges Ergebnis, wenn es in 32 Bit passt
- `mulh`/`mulhu`/`mulhsu` nur nötig, wenn das volle 64-Bit-Ergebnis gebraucht wird

```c
int mul(int a, int b) {
  return a * b;
}

unsigned int mulu(unsigned int a, unsigned int b) {
  return a * b;
}
```

Compiler →

```
mul:
  mul a0, a0, a1
  ret

mulu:
  mul a0, a0, a1
  ret
```

Logische Befehle

- Logisches UND, ODER und XOR sind verfügbar mit 12-Bit-Immediates und Registern

op	funct3	funct7	Type	Instruction	Description	Operation
0010011 (19)	100	–	I	`xori rd, rs1, imm`	xor immediate	`rd = rs1 ^ SignExt(imm)`
0010011 (19)	110	–	I	`ori rd, rs1, imm`	or immediate	`rd = rs1 | SignExt(imm)`
0010011 (19)	111	–	I	`andi rd, rs1, imm`	and immediate	`rd = rs1 & SignExt(imm)`
0110011 (51)	100	0000000	R	`xor rd, rs1, rs2`	xor	`rd = rs1 ^ rs2`
0110011 (51)	110	0000000	R	`or rd, rs1, rs2`	or	`rd = rs1 | rs2`
0110011 (51)	111	0000000	R	`and rd, rs1, rs2`	and	`rd = rs1 & rs2`

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

A	B	$A \vee B$
0	0	0
0	1	1
1	0	1
1	1	1

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Bitmasken

- `and` mit Maske: Bits an den 0-Stellen der Maske löschen
- `or` mit Maske: Bits an den 1-Stellen der Maske setzen
- `xor` mit Maske: Bits an den 1-Stellen der Maske invertieren

```
andi t0, a0, 0b1111
```

a0	1	0	1	1	0	1	1	0
∧ Maske	0	0	0	0	1	1	1	1
= t0	0	0	0	0	0	1	1	0

Schiebebefehle

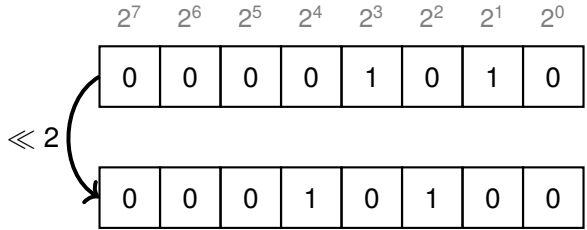
- Logische Shifts um *n*
  - Jedes Bit wird um *n* nach links/rechts geschoben
  - Entstehende Bit-Positionen werden mit Nullen aufgefüllt
- Arithmetischer Rechtsshift um *n*
  - Jedes Bit wird um *n* nach rechts geschoben
  - Kopien des Vorzeichenbits an der Vorzeichenstelle eingeschoben

op	funct3	funct7	Type	Instruction	Description	Operation
0010011 (19)	001	0000000*	I	`slli rd, rs1, uimm`	shift left logical immediate	`rd = rs1 ≪ uimm`
0010011 (19)	101	0000000*	I	`srli rd, rs1, uimm`	shift right logical immediate	`rd = rs1 ≫ uimm`
0010011 (19)	101	0100000*	I	`srai rd, rs1, uimm`	shift right arithmetic immediate	`rd = rs1 ⋙ uimm`
0110011 (51)	001	0000000	R	`sll rd, rs1, rs2`	shift left logical	`rd = rs1 ≪ rs2`$_{4:0}$
0110011 (51)	101	0000000	R	`srl rd, rs1, rs2`	shift right logical	`rd = rs1 ≫ rs2`$_{4:0}$
0110011 (51)	101	0100000	R	`sra rd, rs1, rs2`	shift right arithmetic	`rd = rs1 ⋙ rs2`$_{4:0}$

*Encoded in $\text{instr}_{31:25}$, the upper seven bits of the immediate field

Linksshifts für unsigned Multiplikation

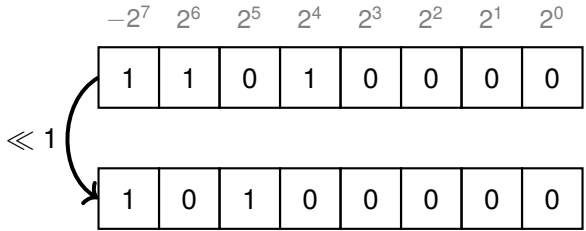
- Logische Shifts um n: Unsigned Multiplikation mit 2^n



- $10 \cdot 4 = 40$
- Funktioniert, weil Multiplikation distributiv ist:
  $2 \cdot (b_0 \cdot 2^0 + b_1 \cdot 2^1 + ...) = 2 \cdot b_0 \cdot 2^0 + 2 \cdot b_1 \cdot 2^1 + ...$

Linksshifts für signed Multiplikation

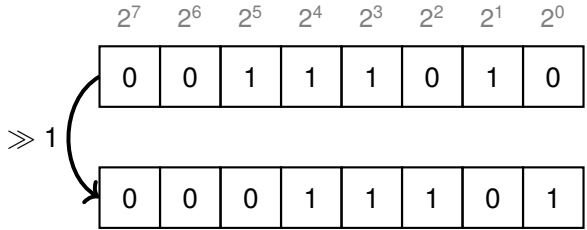
- $1101_0000_2 = -(0010_1111_2 + 1) = -0011_0000_2 = -48$



- Jedes Gewicht verdoppelt sich
- Problem: -2^8 und $+2^7$ gibt es im 8-Bit-Register nicht …
- … aber zusammen schon: $-2^8 + 2^7 = -256 + 128 = -2^7$
  $\Rightarrow -96 = -2^7 + 2^5 = 1010_0000_2$

Rechtsshifts für unsigned Division

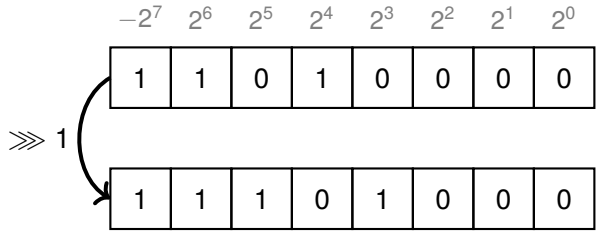
- Logische Rechtsshifts um n: Unsigned Division durch 2^n



- $0011_1010_2 = 58_{10}$
- $0001_1101_2 = 29_{10}$

Rechtsshifts für signed Division

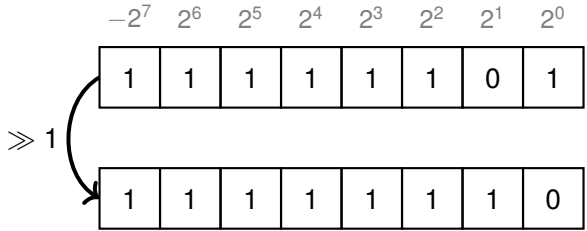
- Arithmetische Rechtsshifts um n: Signed Division durch 2^n



- $1101_0000_2 = -(0010_1111_2 + 1_2) = -0011_0000_2 = -48_{10}$
- $1110_1000_2 = -(0001_0111_2 + 1_2) = -0001_1000_2 = -24_{10}$

Rechtsshifts für signed Division

- $-3 = -(0000_0011_2) = 1111_1100_2 + 1 = 1111_1101_2$
- Aufgabe: $-3/2$



- $1111_1110_2 = -(0000_0001_2 + 1_2) = -0000_0010_2 = -2_{10}$
- Division rundet gegen 0, aber mit Shifts gegen $-\infty$
- Kann aber leicht durch eine Addition behoben werden (Knobelaufgabe)

Pseudobefehle

- Befehle, die nicht direkt als Maschinencode-Anweisung existieren
- Vereinfachte Form, die dann in einen oder mehrere echte Maschinenbefehle umgesetzt wird
- Vorteil: Lesbarer und einfacher zu programmieren
- Beispiele:
  - `mv rd, rs` ist Pseudobefehl für `addi rd, rs, 0`
  - `not rd, rs` ist Pseudobefehl für `xori rd, rs, -1`
  - `neg rd, rs` ist Pseudobefehl für `sub rd, zero, rs`
  - `li rd, imm` ist Pseudobefehl, der 32-Bit-Konstanten lädt

Häufige Fehler

- Davon ausgehen, dass Register null sind
- Vorzeichen bei Operationen beachten
- Zu wenig Gedanken vor dem Programmieren gemacht (wichtig bei Hausaufgabe 2)
- Zu wenig Kommentare im Code
- Schleifen einmal zu oft/zu wenig durchlaufen
- Sign-Extension von Befehlen nicht beachtet (z. B. bei `lb`)
- Ausblick: Calling Convention nicht eingehalten
- **Lösung:** Programme Befehl für Befehl durchgehen und Register überprüfen

QtRVSim

- Simulator für RISC-V Prozessor
- Installationsdateien und Build-Anleitung auf GitHub, Link + Einführung auf Übungsblatt
- Bei Problemen: Suchmaschine, Tutor*innen oder Zulip ERA-Technik Stream befragen
- Einführung in dieser Zentralübung und in den Tutorübungen nächste Woche
- Verwendung für Hausaufgaben notwendig und gute Übung für Klausur
- **Bitte bis zur Übung installieren und Laptop mitbringen**

Beispielaufgaben

- Register `s5` nullen
- Wert in Register `a1` nach `s3` kopieren
- Das unterste Bit von `a0` nach `t1` schreiben, restliche Bits sollen 0 sein
- Register `s1` mit Einsen füllen
- Unterstes Byte von `t0` nach `a0` schreiben, Rest unverändert
- `t4` $\cdot 10$ ohne Multiplikationsbefehle
- Sekunden in `a0` in Stunden(`a0`)/Minuten(`a1`)/Sekunden(`a2`) umrechnen

Quiz

https://partici.fi/93236618

